shake-mqtt

Python bridge that listens for **Raspberry Shake** UDP datagrams and publishes them to **MQTT**. Intended to run on a small server (for example a Raspberry Pi) via **Docker Compose**.

Quick start

1. Copy the environment template and edit values (especially `MQTT_BROKER_HOST` if your broker is not named `mqtt` on the Compose network):

   ```
   cp .env.example .env
   ```

2. Build and run:

   ```
   docker compose up -d --build
   ```

   Plain `docker compose up -d` does **not** rebuild the image; use `--build` (or `make up` in this repo) so Compose builds when the Dockerfile or `shake_mqtt/` changed. Docker still uses layer cache, so unchanged layers are skipped.

3. Point your Shake's UDP output at this machine's IP and the port you set (`SHAKE_UDP_PORT` , default `8888`).

Compose reads the project `.env` file for interpolation in `docker-compose.yml`, and this repo also passes it to the container via `env_file` so bridge settings are available at runtime.

Configuration

Variable	Description	Default
`SHAKE_UDP_BIND_HOST`	Address the UDP socket binds to	`0.0.0.0`
`SHAKE_UDP_PORT`	UDP listen port (also used for host/container port mapping)	`8888`
`SHAKE_UDP_RECV_BUFSIZE`	Maximum datagram size for `recvfrom`	`2048`
`MQTT_BROKER_HOST`	MQTT broker hostname or IP	`mqtt`
`MQTT_BROKER_PORT`	MQTT broker port	`1883`
`MQTT_TOPIC`	Base topic; JSON on `{base}/json`; STA/LTA on `{base}/event/<field>`; catalog on `{base}/match/<field>`	`home/seismic/shake`
`MQTT_CLIENT_ID`	MQTT client id	`shake-bridge`
`MQTT_USERNAME`	Optional broker username	*(empty)*
`MQTT_PASSWORD`	Optional broker password	*(empty)*
`MQTT_TLS`	Use TLS to the broker (`true` / `false`)	`false`

`LOG_LEVEL`	Python logging level (`DEBUG`, `INFO`, …)	`INFO`
`SHAKE_JSON_STRUCTURED`	Publish `/json` as an object (`channel`, `time`, `samples`, stats) instead of a flat array	`false`
`SHAKE_DETECT_EVENTS`	Enable STA/LTA-style detection; publishes leaf topics under `{base}/event/` (e.g. `kind`, `channel`, `ratio`)	`false`
`SHAKE_DETECT_STA_SEC` / `SHAKE_DETECT_LTA_SEC`	Short / long energy window (seconds)	`1` / `30`
`SHAKE_DETECT_RATIO_ON` / `SHAKE_DETECT_RATIO_OFF`	Trigger / reset ratio (hysteresis)	`4` / `2`
`SHAKE_DETECT_COOLDOWN_SEC`	Minimum time after reset before another trigger	`5`
`SHAKE_DETECT_STARTUP_GRACE_SEC`	Suppress new triggers for this many seconds after process start	`5` (`0` = disabled)
`SHAKE_DETECT_DEFAULT_FS_HZ`	Assumed sample rate until inferred from packet spacing	`100`
`SHAKE_DETECT_CHANNELS`	Comma-separated allowlist (empty = all channels)	*(empty)*
`SHAKE_CATALOG_ENABLE`	After each trigger, query USGS and publish leaf topics under `{base}/match/` (e.g. `catalog_present`, `magnitude`, `ref/channel`)	`false`
`SHAKE_CATALOG_LATITUDE` / `SHAKE_CATALOG_LONGITUDE`	Required when catalog is enabled (station location for radius filter and travel-time prediction)	*(unset)*
`SHAKE_CATALOG_MAX_RADIUS_KM`	USGS search radius	`500`
`SHAKE_CATALOG_TIME_BEFORE_SEC` / `SHAKE_CATALOG_TIME_AFTER_SEC`	Catalog time window around trigger	`120` / `60`
`SHAKE_CATALOG_QUERY_OFFSETS_SEC`	Comma-separated seconds after each trigger to run a USGS query (e.g. `60,120,180` for 1, 2, and 3 minutes). Default `60,120,180` when unset. Use `0` alone for one immediate query	`60,120,180`
`SHAKE_CATALOG_USE_TRAVELTIME`	Prefer events whose **predicted first arrival** (origin + model travel time) is closest to the trigger. The USGS catalog API does **not** ship per-station travel times; the bridge calls the	`true`

	IRIS/EQScope traveltime web service (1-D model: `iasp91`, `prem`, or `ak135`). Set `false` to restore origin-time-only matching	
`SHAKE_CATALOG_TRAVELTIME_MODEL`	Velocity model for IRIS (`iasp91` / `prem` / `ak135`)	`iasp91`
`SHAKE_CATALOG_TRAVELTIME_TIMEOUT_SEC`	Per-request timeout for each IRIS lookup (candidates are queried in parallel)	`6`
`SHAKE_CATALOG_TRAVELTIME_MAX_WORKERS`	Max parallel IRIS requests when scoring candidates	`6`
`SHAKE_MATCH_HISTORY_ENABLE`	When catalog is enabled, also publish retained JSON on `{base}/match/history_json` (rolling list for dashboards)	`true`
`SHAKE_MATCH_HISTORY_WINDOW_HOURS`	Drop history rows whose `ref_trigger_time` is older than this many hours	`24`
`SHAKE_MATCH_HISTORY_MAX_ENTRIES`	After time pruning, keep at most this many rows (strongest `sta_rms` first)	`200`

See .env.example for a ready-to-copy template.

If the broker runs in the **same** Compose stack, set `MQTT_BROKER_HOST` to that service's name and add `depends_on` as needed in your compose file. If the broker is **external**, set `MQTT_BROKER_HOST` to its hostname or IP reachable from the container.

Networking notes

- Published UDP (`ports`) is usually enough when the Shake sends to the Pi's LAN address.
- If you depend on host-only broadcast or multicast behavior, you may need `network_mode: host` instead of bridged networking (trades isolation for lower-level network behavior).

Local development (without Docker)

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
export MQTT_BROKER_HOST=127.0.0.1   # example
PYTHONPATH=. python -m shake_mqtt
```

Extending processing

The bridge is split into small pieces so you can swap behavior without rewriting the UDP or MQTT layers:

- **`DatagramProcessor`** (shake_mqtt/processing.py) — `process(...) -> ProcessResult` with optional `json_payload` for **`{MQTT_TOPIC}/json`** and optional `event_payloads` (internal JSON strings) for STA/LTA. **`ShakeMqttBridge`** publishes those as **MQTT leaf topics** under **`{MQTT_TOPIC}/event/<field>`** (only for **`kind: trigger`**), not a single JSON blob on `{base}/event`. Parsing handles Raspberry Shake UDP lines like `{"CHANNEL", timestamp, sample, ...}` (UTF-8, `'` → `"`, outer `{…}` → JSON array or structured object). Invalid datagrams yield an empty result (nothing published).
- **`build_processor(config)`** — returns **`PassthroughJsonNormalizer`** or **`EventDetectingProcessor`** (STA/LTA) based on `SHAKE_DETECT_EVENTS`. Detection uses **AC energy** (squared deviation from a slow exponential DC), so **`sta_rms`** / **`lta_rms`** are on a similar scale to **`packet_rms`**, not raw digitizer counts. With detection on, each **`/json`** message includes **`sta_rms`** and **`lta_rms`** (structured object keys, or two extra trailing numbers in the flat array). Changing this algorithm can shift optimal **`SHAKE_DETECT_RATIO_*`** values.
- **`ShakeMqttBridge`** (shake_mqtt/bridge.py) — accepts an optional `processor=` argument for dependency injection; when **`SHAKE_CATALOG_ENABLE`** is set, schedules one USGS lookup per entry in **`SHAKE_CATALOG_QUERY_OFFSETS_SEC`** (default **1, 2, and 3 minutes** after the trigger) and publishes **leaf topics** under **`{MQTT_TOPIC}/match/`** after each attempt (see `shake_mqtt/topic_publish.py`). With travel time enabled, the chosen event minimizes the gap between the trigger time and **origin time + IRIS first-arrival travel time**; **`sta_rms`** still comes from the trigger.
- **Match history** (shake_mqtt/match_history.py) — when catalog and **`SHAKE_MATCH_HISTORY_ENABLE`** are on, each completed lookup updates **`{MQTT_TOPIC}/match/history_json`**: retained JSON `{"matches": [...],"updated_unix":...}`. Catalog fields mirror MQTT leaf topics; **`sta_rms`** is always the trigger value (so sorts and plots work when USGS returns no row, unlike **`Shake/match/sta_rms`**, which is empty then). **`matches`** is sorted by **`sta_rms`** (descending). Example MQTT sensor + Markdown dashboard table: homeassistant/shake_event_template.yaml (comments) and the snippet below.

Lovelace Flex Table card (table, history rows from `matches`) — this is more robust for array attributes than Markdown loops.

Use the ready-to-paste card YAML at: `homeassistant/shake_match_history_flex_table_card.yaml`

Install `custom:flex-table-card` via HACS (or manual resource) before using this card.

If rows are empty, first verify the exact sensor entity id in Home Assistant (for example, `sensor.shake_shake_match_history` vs `sensor.shake_match_history`) and update the card `entities` entry accordingly.

Example direction: implement `DatagramProcessor` with your own `ProcessResult` to add filters or extra MQTT topics.

License

See LICENSE.